

Ask Rammy

WCU HR AI Chatbot — Project Documentation

West Chester University | CSC402 | 2026

1. Project Overview

Ask Rammy is an AI-powered HR chatbot for West Chester University employees. Built as a capstone project for CSC402, it allows employees to ask natural language HR questions and receive accurate, sourced answers drawn from official WCU and PASSHE documentation.

Rammy answers HR questions about benefits, retirement, payroll, parking, FMLA, workers' comp, tuition waivers, I-9 documentation, and more — all sourced from official WCU and PASSHE pages.

1.1 Key Features

- Natural language HR Q&A powered by OpenAI GPT-4.1-mini
- Knowledge retrieval from 20+ official WCU and PASSHE source pages
- PII detection — blocks submission of personal information
- Clickable source links in responses when more detail is available
- GPT-generated varied out-of-scope declines in Rammy's friendly voice
- Conversation memory — retains last 4 turns of context
- Message timestamps on every chat bubble
- Live status dot — green/red indicator showing backend connectivity
- Minimize to bubble — close button collapses chat to a floating launcher
- Options dropdown — clear conversation, refresh sources, contact HR
- Fully containerized with Docker for one-command startup
- REST API architecture — frontend never touches the OpenAI API key

2. Architecture

The application is a three-layer stack running inside Docker. The browser communicates only with the Node.js server, which forwards requests to the Python backend. The OpenAI API key is never exposed to the client.

Layer	Technology & Role
Frontend	HTML + Vanilla JS (chat.js) — Chat UI, timestamps, status dot, minimize, options dropdown
API Server	Node.js Express (port 3000) — CORS, rate limiting, input validation, request forwarding
AI Backend	Python Flask (port 5001) — PII detection, knowledge retrieval, prompt building, OpenAI call
AI Model	OpenAI GPT-4.1-mini — Generates answers and out-of-scope declines
Knowledge Base	20+ scraped WCU/PASSHE HR pages — chunked and scored at query time
Container	Docker Compose — Orchestrates Python and Node services together

2.1 Request Flow

- User types a question in the browser
- chat.js POSTs to Node.js at /api/chat with the message and conversation history
- Node.js validates the request, trims history, and forwards to Python /chat
- Python checks for PII — blocks if found
- Python detects small talk or identity questions and short-circuits with a canned response
- For HR questions, Python scores all chunks and retrieves the top 5 most relevant
- Python builds a system prompt with the retrieved context and source URLs
- GPT-4.1-mini generates a natural answer, including an HTML link if relevant
- If no context is found, GPT generates a friendly Rammy-style decline
- The reply travels back through Node.js to the browser and renders in the chat bubble

3. File Structure

File	Purpose
chatbot_api.py	Python Flask backend — all AI and retrieval logic
server.js	Node.js Express REST API server
chat.js	Frontend JavaScript — UI, API calls, conversation history, timestamps, status dot, minimize/restore, options dropdown
index.html	Chat UI markup
styling.css	WCU-branded styles (purple/yellow color scheme)
Dockerfile.python	Docker image definition for the Python service
Dockerfile.node	Docker image definition for the Node.js service
docker-compose.yml	Orchestrates both containers with one command
.env	API keys and environment config (never committed to Git)
requirements.txt	Python dependencies
package.json	Node.js dependencies
INTEGRATION_PLAN.md	Full API reference and setup guide
STARTUP.md	Quick-start commands for daily use
README.md	Repository overview and Docker setup instructions
architecture.svg	Architecture diagram (renders on GitHub)

4. Frontend Features

4.1 Status Dot

A small circle sits next to the 'Ask Rammy' title in the header. It starts gray on load, turns green when both the Node.js server and Python backend are reachable, and turns red if either is unavailable. The status is checked on page load and automatically rechecked every 30 seconds.

4.2 Close Button — Minimize to Bubble

Clicking the X button in the header collapses the chat window entirely and replaces it with a small purple floating bubble in the bottom-right corner of the page. Clicking the bubble restores the full chat window. Conversation history is preserved during minimize/restore.

4.3 Options Dropdown

Clicking the  button opens a dropdown menu with three actions:

- Clear conversation — wipes all messages and history after a confirmation prompt, then shows the welcome message again
- Refresh HR sources — calls the `/api/refresh` endpoint to reload all HR pages in the background
- Contact HR — displays Rammy's message with a clickable email link (`HRS@wcupa.edu`) and phone link (610-436-2800)

The dropdown closes automatically when clicking outside of it.

4.4 Message Timestamps

Every chat bubble displays a timestamp showing the time the message was sent, formatted as 12-hour time with AM/PM (e.g. 2:45 PM). Timestamps appear next to the name tag below each bubble.

4.5 Clickable Source Links

When Rammy's answer is partial or would benefit from more detail, the response includes a clickable HTML anchor link pointing to the relevant WCU or PASSHE source page. Links open in a new tab and are styled in WCU purple. Raw URLs never appear — only natural anchor text like 'Learn more about retirement plans here'.

5. Setup & Running

5.1 Prerequisites

- Docker Desktop — docker.com/products/docker-desktop
- Git — git-scm.com
- OpenAI API key — platform.openai.com
- VS Code with Live Server extension (for the frontend)

5.2 First-Time Setup

Clone the repository

```
git clone -b Garrett_Crowner https://github.com/GarrettCrowner/CSC402-Project.git
cd CSC402-Project
```

Create the .env file

```
echo 'OPENAI_API_KEY=sk-your-key-here' > .env
```

Never commit your .env file to Git. Your API key is private and should never be shared.

5.3 Daily Startup

Start Docker Desktop, then run:

```
docker-compose up --build # first time or after code changes
docker-compose up         # every time after
```

Open the frontend

Right-click index.html in VS Code and select Open with Live Server, or run:

```
open index.html
```

Verify everything is connected

```
curl http://localhost:3000/api/health
```

Expected: { "status": "ok", "python": "reachable" }

5.4 Shutdown

```
docker-compose down
```

6. Knowledge Base

Rammy's knowledge comes entirely from scraping official WCU and PASSHE HR pages at startup. Pages are chunked into ~700 character segments and scored at query time using keyword matching and phrase boosts.

6.1 Source Pages

Category	Source URL
HR FAQs	wcupa.edu/hr/faqs.aspx
FMLA Leave	wcupa.edu/hr/FMLA.aspx
Employee Relations	wcupa.edu/hr/employee-labor-relations.aspx
Benefits by Group	wcupa.edu/hr/employee-benefits-vs-benefits-by-employee-group.aspx
Tuition Waiver	wcupa.edu/hr/tuition-waiver.aspx
Workers Comp	wcupa.edu/hr/work-related-injuries.aspx
Payroll	wcupa.edu/_information/AFA/fbs/payroll.aspx
Parking Permits	wcupa.edu/dps/parkingservices/parkingPermits.aspx
Parking FAQs	wcupa.edu/dps/parkingservices/faqs.aspx
I-9 Documents	uscis.gov/i-9-central/form-i-9-acceptable-documents
Retirement Index	passhe.edu/hr/benefits/retirement/index.html
Voluntary Retirement	passhe.edu/hr/benefits/retirement/voluntary-retirement-plans.html
TSA 403(b)	passhe.edu/hr/benefits/retirement/tsa.html
Deferred Comp 457	passhe.edu/hr/benefits/retirement/deferred-compensation.html
ARP Plan	passhe.edu/hr/benefits/retirement/arp.html
SERS	passhe.edu/hr/benefits/retirement/sers.html

6.2 Adding New Sources

To expand Rammy's knowledge, add a URL to the `ALLOWED_URLS` list in `chatbot_api.py`:

```
ALLOWED_URLS = [  
    ...existing URLs...  
    "https://www.wcupa.edu/hr/your-new-page.aspx",  
]
```

Then add relevant phrase boosts to the `PHRASE_BOOSTS` list for better keyword matching, and add any synonym normalizations to `_QUERY_REPLACEMENTS`.

6.3 Refreshing Sources

Sources are loaded once at startup. To reload without restarting the container, type `/refresh` in the chat or call the refresh endpoint:

```
curl -X POST http://localhost:3000/api/refresh
```


7. API Reference

POST /api/chat

Sends a user message and returns Rammy's reply.

Request

```
{ "message": "How do I apply for FMLA?",  
  "history": [{ "role": "user", "content": "Hi" },  
               { "role": "assistant", "content": "Hey there!" }] }
```

Response

```
{ "reply": "To apply for FMLA, you need to..." }
```

POST /api/refresh

Triggers a background reload of all HR source pages.

```
{ "message": "Source refresh started in background." }
```

GET /api/health

Liveness check — also pings the Python service.

```
{ "status": "ok", "python": "reachable" }
```

8. PII Detection

Rammy checks every user message for personally identifiable information before processing it. If PII is detected, the message is blocked and the user is prompted to remove it.

PII Type	Pattern Detected
Email addresses	Standard email format (user@domain.com)
Phone numbers	US phone formats including country code
Social Security Numbers	XXX-XX-XXXX format
Street addresses	Number + street name + type (St, Ave, Rd, etc.)
Name introductions	"My name is..." / "This is..."
Long ID numbers	Any 6+ digit numeric string
Bank/card numbers	13-16 digit numeric sequences

9. Troubleshooting

Symptom	Fix
Connection refused on port 3000	Start Docker Desktop, then run <code>docker-compose up</code>
"python unreachable" in health check	Run <code>docker logs rammy-python</code> to see the error
Port 5000 already in use	Mac AirPlay uses 5000 — the app uses 5001 to avoid this
.env not found error	Create the <code>.env</code> file with your OpenAI API key in the repo root
Rammy says 'I cannot answer'	The topic may not be in the knowledge base — try <code>/refresh</code> or add a source URL
Link not clickable in chat	Make sure you are using the updated <code>chat.js</code> with <code>sanitizeHtml</code>
Changes not reflected	Run <code>docker-compose down</code> then <code>docker-compose up --build</code>

10. Recommended Next Steps

9.1 Cloud Deployment

The project is Docker-ready and can be deployed to any cloud platform that accepts containers. Recommended options:

- Render — free tier available, supports Docker, easy GitHub integration
- Railway — simple container deployment with automatic HTTPS
- AWS ECS or Azure Container Apps — for enterprise-grade scaling

Before deploying publicly, move the `OPENAI_API_KEY` to the cloud platform's secret manager rather than a `.env` file.

9.2 Semantic Search with Embeddings

The current keyword scorer works well but misses questions phrased differently from the source text. Replacing it with OpenAI embeddings would dramatically improve answer quality:

- Embed all chunks at startup using `text-embedding-3-small`
- Embed each user query at query time
- Use cosine similarity instead of keyword matching to find relevant chunks
- This would allow 'hurt at work' to match 'workplace injury' without explicit synonym rules

9.3 Rammy Profile Picture

Add Rammy's mascot image to the chat header for a more polished experience. In `index.html`, add an `<img>` tag inside the header and style it in `styling.css` with a circular crop and WCU yellow border.

9.4 Scheduled Source Refresh

Sources are currently only refreshed manually. Add `APScheduler` to `chatbot_api.py` to automatically reload pages nightly:

```
from apscheduler.schedulers.background import BackgroundScheduler
scheduler = BackgroundScheduler()
scheduler.add_job(_load_sources, 'cron', hour=2) # 2am daily
scheduler.start()
```

9.5 Session-Based Conversation History

Currently conversation history is stored client-side and sent with every request. For a more robust solution, add session IDs so the Node server maintains per-user history server-side, reducing payload size and enabling history persistence across page reloads.

9.6 React Frontend Migration

The React version of the frontend (chatReact.js) is already written and ready to wire up. Migrating to React would enable response streaming — where Rammy's answer appears word by word like ChatGPT — using the OpenAI streaming API and Server-Sent Events.

9.7 Production WSGI Server

Flask's built-in server is not suitable for production traffic. Replace it with Gunicorn by updating Dockerfile.python:

```
pip install gunicorn
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5001", "chatbot_api:app"]
```

11. HR Contact Information

For questions that fall outside Rammy's scope, employees should contact WCU HR directly:

Contact Method	Details
Email	HRS@wcupa.edu
Phone	610-436-2800
Office	201 Carter Drive, Suite 100, West Chester, PA 19383
Website	wcupa.edu/hr